

SQL Injection

Application: Damn Vulnerable Web Application (DVWA)

Environment: Ubuntu – Apache – MySQL

Vulnerability Type: SQL Injection

Security Levels Tested: Low, Medium, High, Impossible

1. Objective

The objective of this lab is to analyze the behavior of SQL Injection vulnerabilities across different security levels in DVWA. The goal is to understand why exploitation succeeds or fails at each level and to evaluate the effectiveness of the implemented security controls through application behavior, server logs, and source code analysis.

2. Environment Setup

- Operating System: Ubuntu
- Web Server: Apache2
- Database: MySQL
- Vulnerable Application: Damn Vulnerable Web Application (DVWA)

3. Low Security Level

Description

At the Low security level, user input is directly concatenated into the SQL query without any form of validation, sanitization, or parameterization.

Code Analysis

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

- User input is directly embedded in the SQL query
- No validation or filtering is applied
- No prepared statements are used

Exploitation

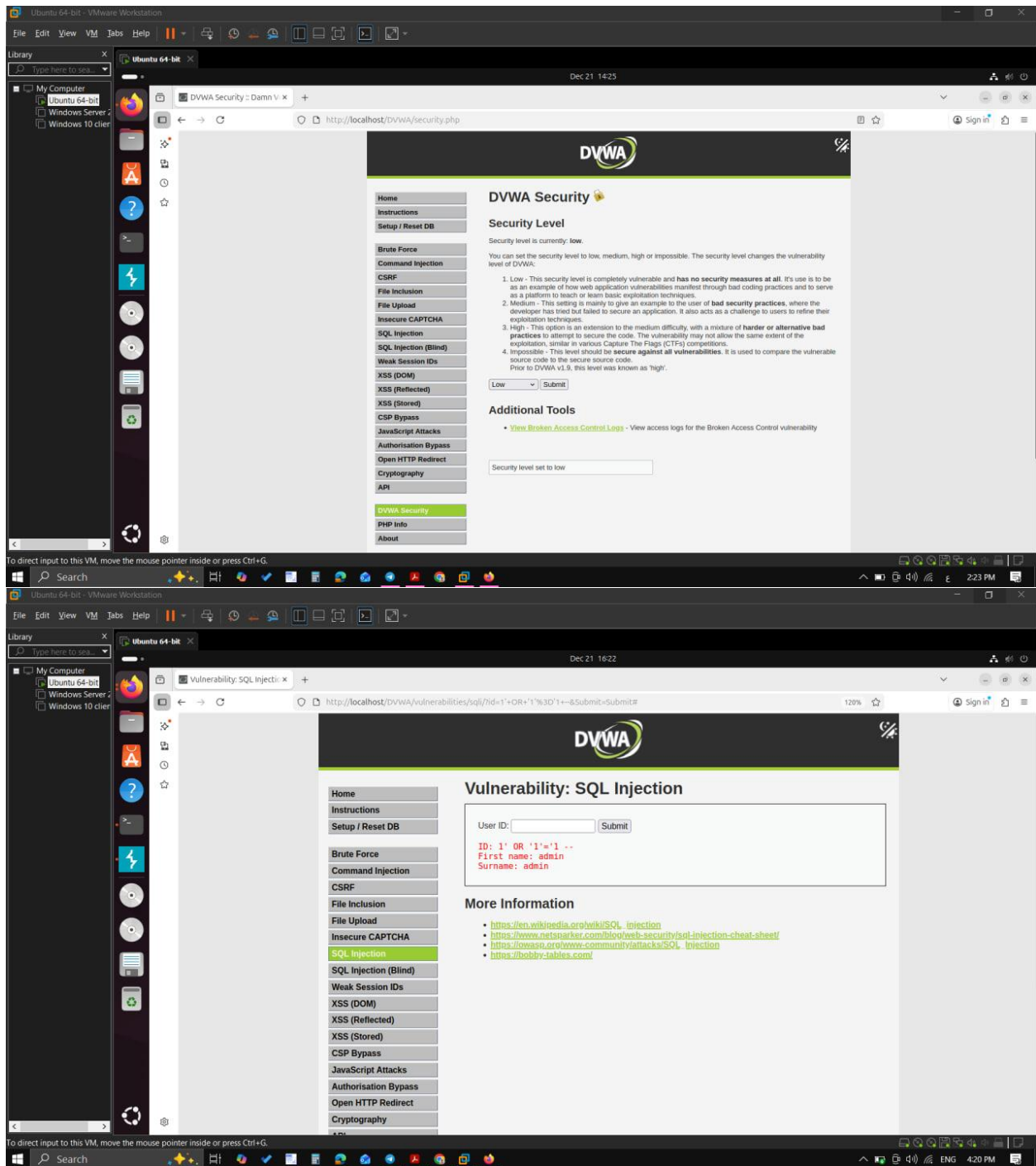
Payload used:

```
1' OR '1'='1
```

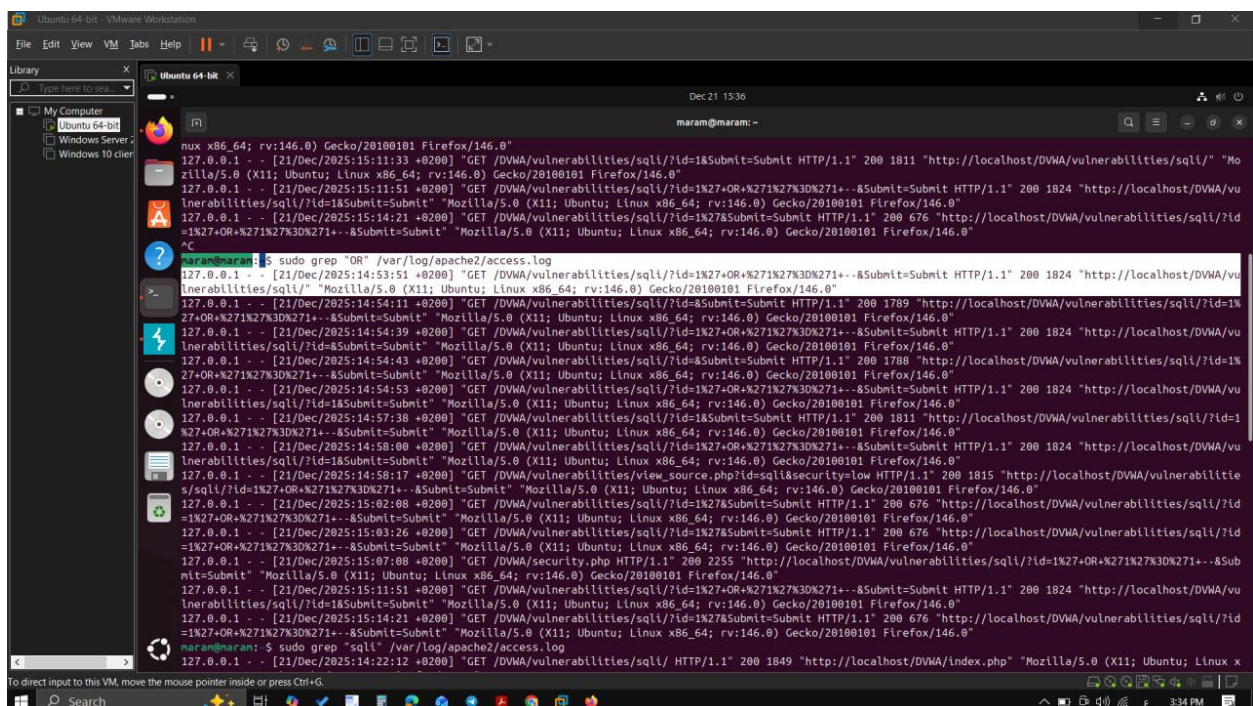
Result

- The SQL query logic is altered
- The condition becomes always true
- Multiple user records are returned

✓ SQL Injection is **successfully exploited**



The Apache access log clearly shows the injected payload inside the request URL. This makes detection straightforward.



4. Medium Security Level

Description

At the Medium security level, the application attempts to mitigate SQL Injection by escaping special characters. However, the SQL query is still dynamically constructed using user input.

Code Analysis

```
$id = mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $id);  
$query = "SELECT first_name, last_name FROM users WHERE user_id = $id;"
```

Why Injection Is Partially Mitigated

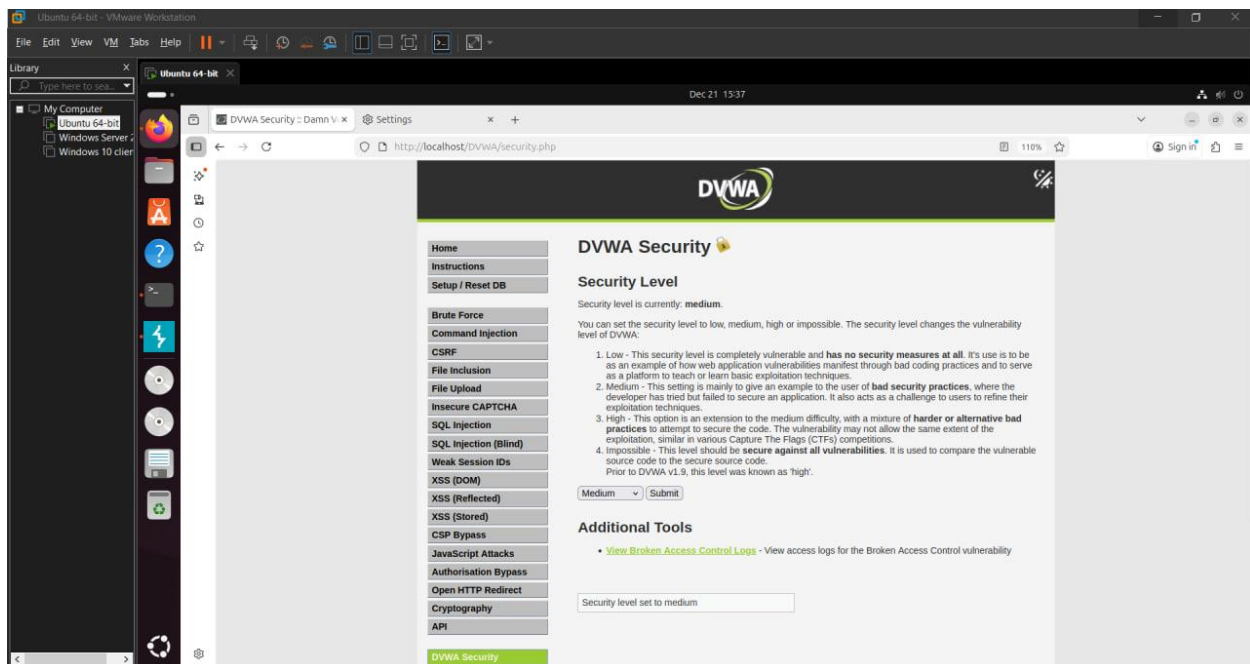
- `mysqli_real_escape_string` prevents basic quote-based payloads
- However:
 - Input is not strictly validated as numeric
 - Prepared statements are not used
 - User input is still directly concatenated into the query

This results in **incomplete protection**.

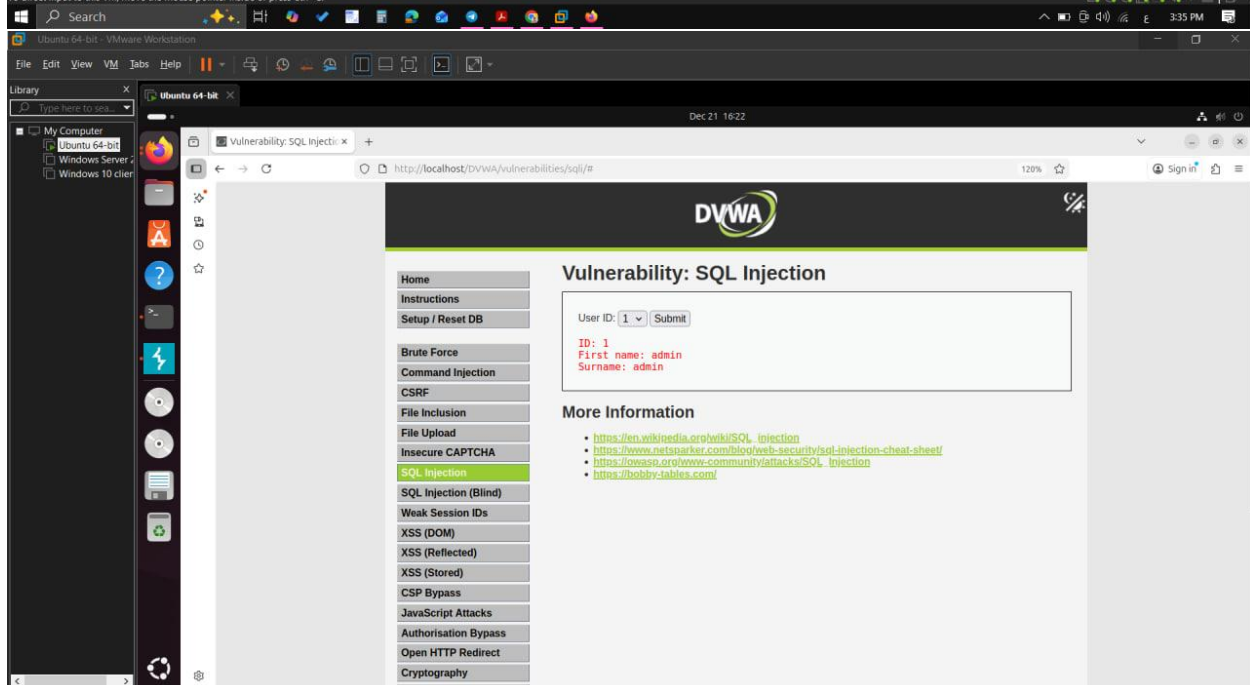
Exploitation Result

- Simple payloads fail
- Advanced numeric-based SQL injection techniques may still be possible
- Application behavior appears normal

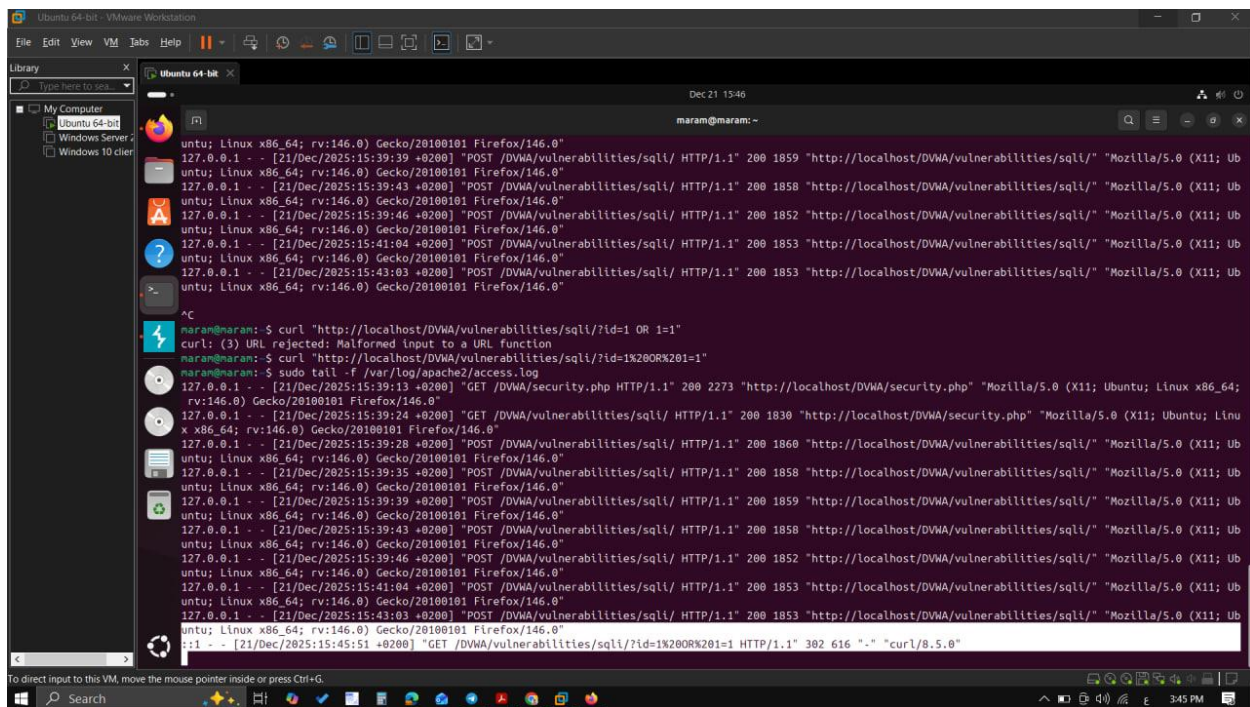
 SQL Injection is **not clearly exploitable**, but the design remains insecure



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.



To direct input to this VM, move the mouse pointer inside or press Ctrl+G.



Security Assessment

This level provides a false sense of security. While basic SQL injection attempts are blocked, the application remains vulnerable to more advanced techniques due to the lack of parameterized queries.

5. High Security Level

Description

At the High security level, the application no longer accepts user input directly from the request. Instead, the user ID is retrieved from a server-side session variable.

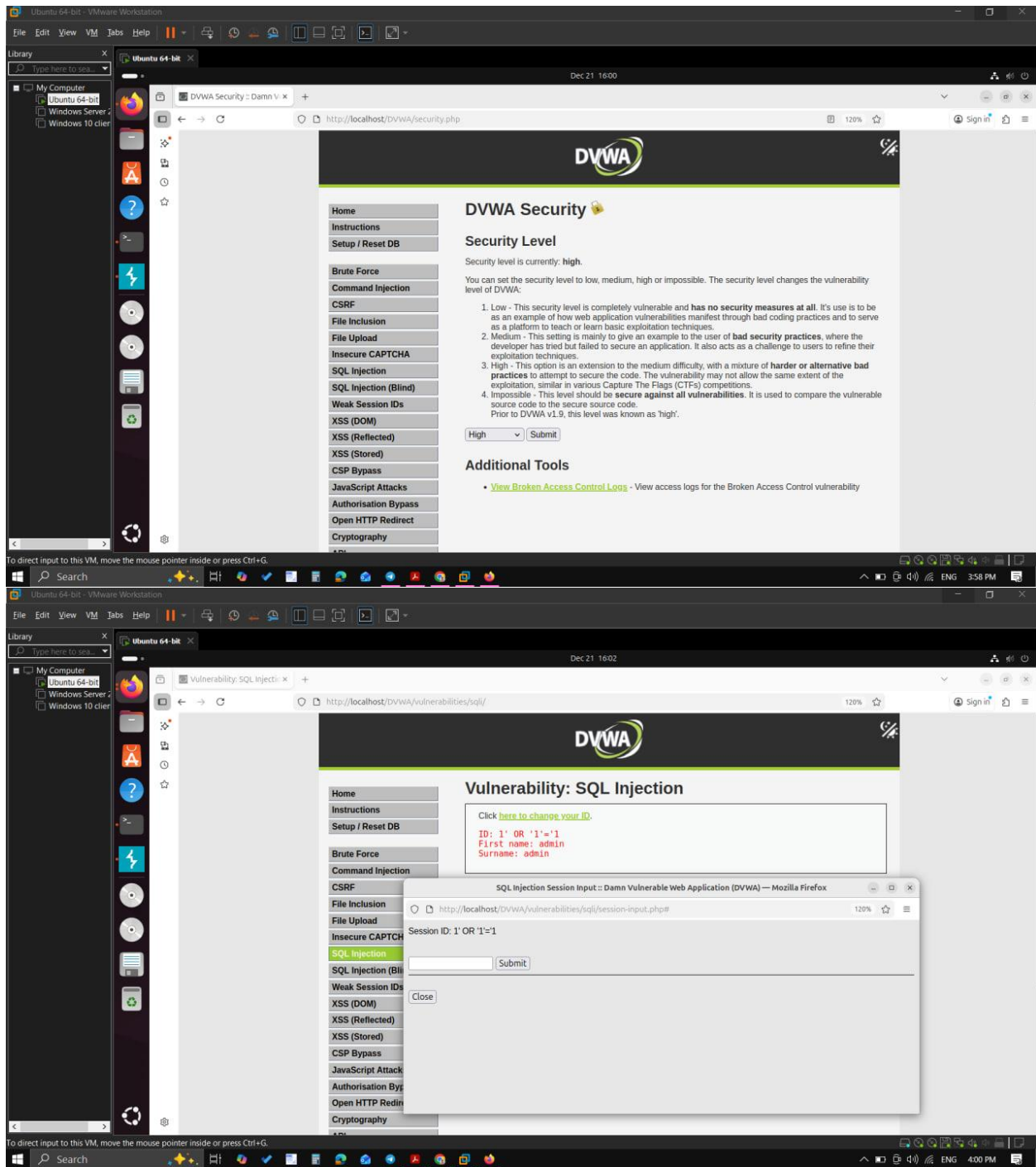
Code Analysis

```
$id = $_SESSION['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;";
```

Why Injection Fails

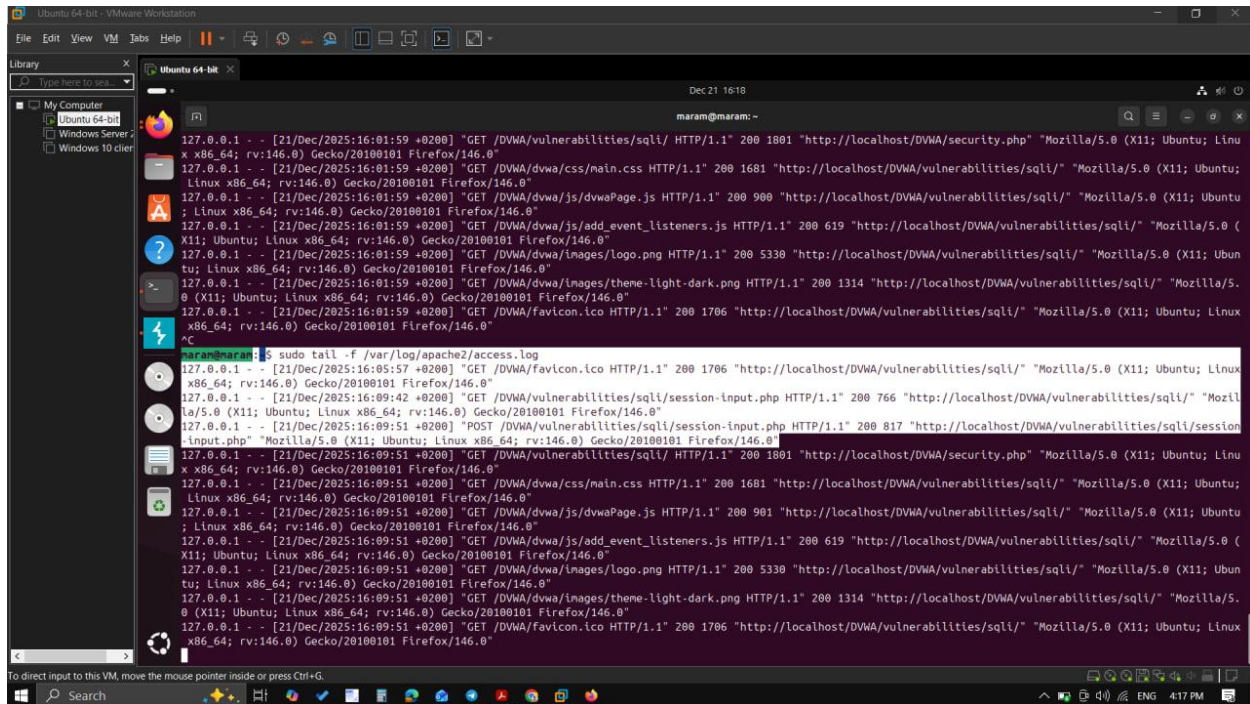
- The attacker cannot directly control the `$id` variable
- The value is managed by application logic
- `LIMIT 1` restricts query results
- No user-controlled injection point exists

✗ SQL Injection is **not** exploitable



Detection

- Server logs show only normal application traffic
- No SQL manipulation patterns appear



6. Impossible Security Level

Description

The Impossible security level demonstrates a secure and correct implementation that fully protects against SQL Injection.

Secure Code Analysis

```
if (is_numeric($id)) {
    $data = $db->prepare(
        'SELECT first_name, last_name FROM users WHERE user_id = (:id) LIMIT
1;')
    );
    $data->bindParam(':id', $id, PDO::PARAM_INT);
}
```

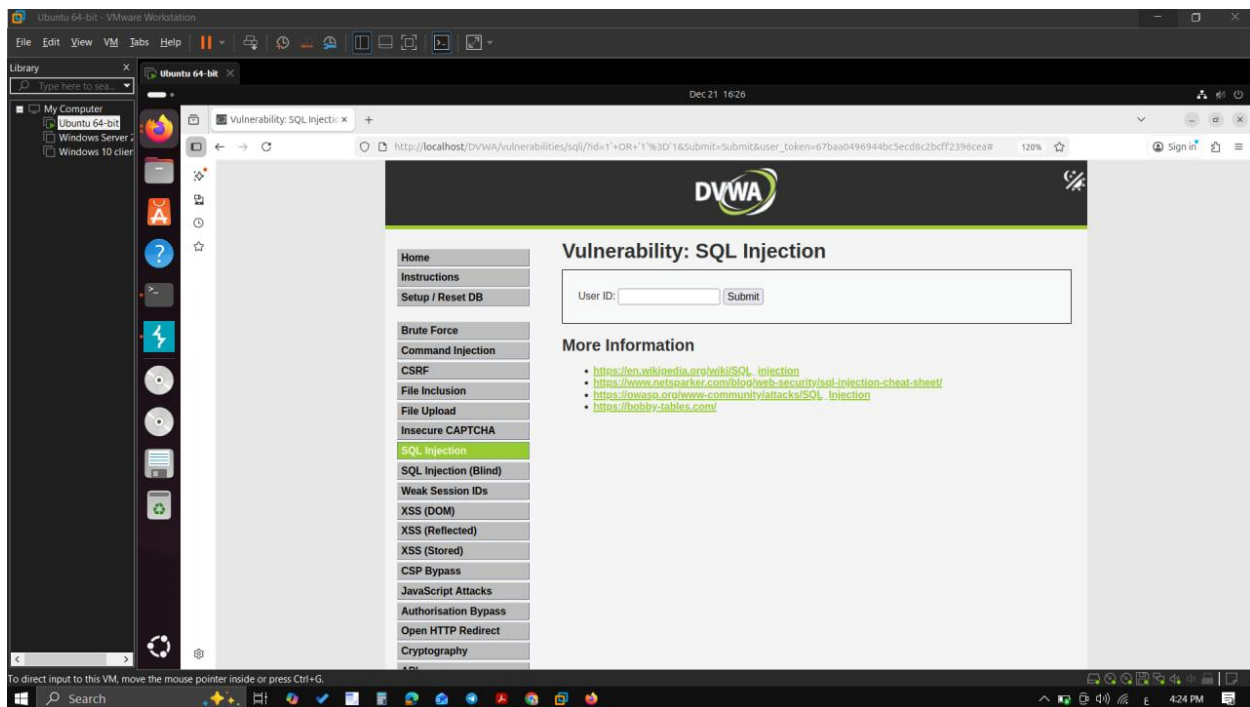
Why Injection Is Impossible

- Strict input validation (`is_numeric`)
- Type casting to integer
- Prepared statements
- Parameter binding

- Anti-CSRF token enforcement

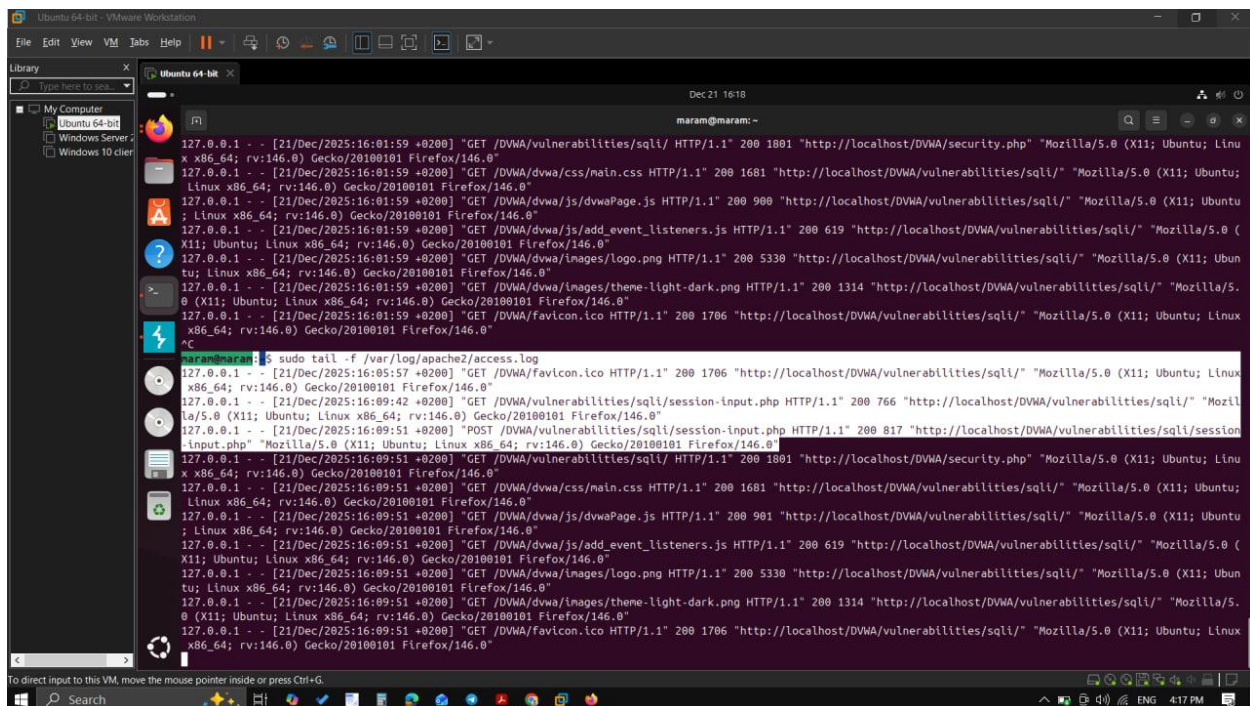
✓ SQL Injection is fully prevented

The image displays two screenshots of a DVWA (Damn Vulnerable Web Application) running in a VMware Workstation environment. The top screenshot shows the 'Vulnerability: SQL Injection' page. The URL bar indicates the application is running on localhost. The page features a sidebar with various vulnerability categories, and the 'SQL Injection' category is selected. The main content area shows a 'User ID' input field with the value '1' OR '1'=1' and a 'Submit' button. Below the input field, there is a 'More Information' section with links to external resources. The bottom screenshot shows the 'DVWA Security' page. The URL bar indicates the application is running on localhost. The page features a sidebar with various vulnerability categories, and the 'SQL Injection' category is selected. The main content area shows the 'Security Level' section, which indicates the current security level is 'Impossible'. Below this, there is a list of security levels and their descriptions. The 'Additional Tools' section includes a link to 'View Broken Access Control Logs'.



Detection

- No malicious payloads appear in logs
- All queries are executed securely
- Application behavior is consistent and safe



7. Comparative Summary

Security Level	Protection Mechanism	Injection Result
Low	None	Fully exploitable
Medium	Input escaping only	Partially mitigated
High	Session-based input	Not exploitable
Impossible	Prepared statements + validation	Fully secure

8. Conclusion

This lab demonstrates how improper handling of user input leads to SQL Injection vulnerabilities and how incremental security improvements reduce the attack surface. Only the Impossible security level implements industry best practices by using parameterized queries and strict input validation, effectively eliminating SQL Injection risks.